

1. Mobile Recharge System

Scenario: A telecom company processes mobile recharges.

Create a **Recharge** class with private variables:

- mobileNumber
- operatorName
- rechargeAmount

Requirements:

- Recharge amount should be at least ₹10.
- Mobile number should contain exactly 10 digits.
- Use getter and setter methods.

2. Food Delivery App

Scenario: A food delivery application manages restaurant orders.

Create an **Order** class with private variables:

- orderId
- customerName
- totalAmount
- orderStatus

Requirements:

- Total amount cannot be negative.
- Status should accept only "Pending", "Preparing", "Delivered", or "Cancelled".
- Use getter and setter methods.

3. Imagine you are developing a Car Rental System, and you need to design a class

named Car to represent individual cars within the system. The Car class should have attributes

model and year. Additionally, you need methods to get and set these attributes.

Attributes:

model: Represents the specific model of the car (e.g., Camry, Accord).

year: Represents the manufacturing year of the car.

Methods:

getModel(): Returns the model of the car.

getYear(): Returns the manufacturing year of the car.

setModel(String model): Sets the model of the car.

setYear(int year): Sets the manufacturing year of the car.

Testcase—1

Input

Camry

2020

Output

Model: Camry

Year: 2020

Explanation

- The model is set to Camry.
- The year is set to 2020.
- getModel() returns Camry.
- getYear() returns 2020.

Testcase–2

Input

Accord

2018

Output

Model: Accord

Year: 2018

Explanation

- The model is Accord, so getModel() returns Accord.
- The manufacturing year is 2018, so getYear() returns 2018.

Testcase–3

Input

Tesla Model 3

2023

Output

Model: Tesla Model 3

Year: 2023

Explanation

- setModel("Tesla Model 3") updates the model successfully.
- setYear(2023) updates the year.
- The getters return these updated values.

4. At CityLibrary, librarian Emma is developing a Java program to calculate fines for overdue books.

She needs a class with:

- **Private variables for bookID and overdueDays**
- **Getters and setters to access and modify values safely**
- **Methods to compute three types of fines:**

Fine Methods:

1. **fineByDays(int days) → days × 5**
2. **fixedFine() → always 10**
3. **fineByDaysPlusFixed(int days) → days × 5 + 10**

Additional Requirement:

- All fines must be capped at 1000 (maximum limit)

This ensures billing does not overflow or produce unrealistic values during periods of high overdue returns.

Input Format

First line : bookID (integer)

Second line : overdueDays (integer)

Third line : days2 → for fineByDays()

Fourth line : dummy integer → for fixedFine()

Fifth line : days3 → for fineByDaysPlusFixed()

Output Format

Output 1: fineByDays(days2)

Output 2: fixedFine()

Output 3: fineByDaysPlusFixed(days3)

Output 4: bookID (via getter)

Output 5: overdueDays (via getter)

Sample TestCase–1

➤ Input

101

10

5

0

3

➤ Output

25

10

25

101

10

Sample TestCase–2

➤ Input

200

20

10

0

5

➤ Output

50

10

35

200

20

Sample TestCase–3

➤ Input

300

50

20

0

10

➤ Output

100

10

60

300

50

5. OTT Subscription Platform

An OTT platform like Netflix or Amazon Prime stores subscriber information such as subscriber ID, name, email, subscription plan, and subscription expiry date. Users should not be allowed to change their subscription details directly. Any changes should be validated before updating the data. Implement this system using

Encapsulation.